

GarudaESE (EV60Y51A) Firmware Porting Plan — Plan A: 6-Step Only

Target board: GarudaESE / EV60Y51A V1.0 — dsPIC33AK256MC506-E/M7 (64-pin VQFN) + ATA6847 gate driver + BSC028N06NSSG MOSFETs, 25 V / 6S ceiling.
Source firmwares: - [garuda-ak-ata-esc](#) (dsPIC33AK128MC106 + ATA6847L, **proven ATA/SPI driver + ATA-comparator ZC**) — borrow the driver layer. - [ProjectGaruda/dspic33AKESC](#) (the tuned AK512/MCLV 6-step tree — **borrow the ADC-ZC / hwzc / sector-PI / GSP / FOC-param machinery**).

Scope of Plan A: Bring up **6-step trapezoidal sensorless only**. FOC is deferred — but the build keeps the FOC parameter set live because the 6-step code already consumes several FOC constants (N/K_e , pole-pairs, Vbus scale). The HAL and parameter layout are arranged so FOC can be layered on later **without re-reporting**.

Companion doc: [GarudaESE_PinMap.md](#) (full schematic pin/passive reference). This plan references it; read both together.

0. Reuse matrix — what comes from where

Layer	Source	File(s)	Action
SPI master (ATA framing)	ata-esc	<code>hal_spi.c/.h</code>	copy verbatim; re-pin PPS only
ATA6847 register driver	ata-esc	<code>hal_ata6847.c/.h</code>	copy; revisit GDUCL1.BEMFEN + CSCR for 506 sensing
ATA boot/start/stop sequencing	ata-esc	<code>garuda_service.c</code> (StartMotor/StopMotor)	copy the GDU enter/standby + bootstrap-charge flow
PWM 6-step (active-low HS, override commutation)	ata-esc and AKESC	<code>hal_pwm.c</code>	merge: ata-esc has the active-low/internal-DT polarity already correct
ADC + BEMF/ZC engine	AKESC	<code>hal_adc.c</code> , <code>motor/bemf_zc.c</code> , <code>motor/hwzc.c</code> , <code>hal_comparator.c</code>	port + extend for 3 dedicated BEMF channels + selectable neutral
Sector-PI / commutation timing	AKESC (<code>hwzc.c</code>) or ata-esc (<code>sector_pi.c</code>)	—	Decision §6: keep AKESC hwzc PLL (more mature)
GSP protocol / params / EEPROM	AKESC	<code>gsp/*</code> , <code>eeeprom.c</code>	copy; add board-id + ATA diag command
Motor/board constants	AKESC	extract from <code>garuda_foc_params.h</code> → new <code>motor_profile.h</code>	clean 6-step: drop the FOC param file, keep only the ~4 motor/board constants 6-step actually uses (§6)
Pin/PPS map	NEW	<code>port_config.c/.h</code>	author for AK256MC506 64-pin per §2

1. Device delta — AK128MC106 (source) → AK256MC506 (target)

Item	AK128MC106 (DIM)	AK256MC506 (GarudaESE)	Impact
Package	80/100-pin DIM	64-pin VQFN	fewer pins, different PPS map
Flash	128 KB	256 KB	linker script / device header swap
ADC modules	2 (ADC1, ADC2)	2 (ADC1, ADC2) — same IP	mux logic re-planned (§4)
ADC channels (S/H + comparator each)	many; A/C shared on AD2CH0	enough free AN pins → 3 dedicated BEMF channels	no A/C mux, no settle penalty
SPI modules	3	3	use SPI2 (board routes SCK2/SDI2/SDO2)
PWM generators	PG1..PGn (fixed H/L pins)	PG1/PG2/PG3 fixed pins (RD2/3, RD0/1, RC3/4)	same physical pins — lucky parity
BEMF sense path	ATA internal comparators → GPIO	external 24k/2k dividers + real virtual-neutral pin + ATA comparators (test-points)	enables ADC-ZC options (§5)

Header/linker: swap to `p33AK256MC506.h` / `33AK256MC506` config; set `FCY` /clock chain per Y1 = 8 MHz MEMS (OSC1/CLKI pin 40) + PLL. Confirm `FOSC` / `FCY` macros match the AKESC tree's clock.c.

2. Complete pin map — dsPIC33AK256MC506 (64-VQFN) on GarudaESE

PWM and most analog are **fixed-function**; SPI/UART/CAN/DShot use **PPS (RPN)**. Pins marked **(confirm)** were below schematic-render resolution — verify on the symbol/silk at integration (see [GarudaESE_PinMap.md](#)).

2.1 Motor PWM → ATA6847 (fixed pins; active-LOW high-side, dead-time internal to ATA)

dsPIC pin	port	PWM gen	→ ATA6847	TRIS
51	RD2	PG1H (PWM1H)	NBH1 (36)	out
52	RD3	PG1L (PWM1L)	IL1 (35)	out
49	RD0	PG2H (PWM2H)	NBH2	out
50	RD1	PG2L (PWM2L)	IL2 (37)	out
43	RC3	PG3H (PWM3H)	NBH3 (40)	out
44	RC4	PG3L (PWM3L)	IL3 (39)	out

PWM-gen → phase order **PG1=U, PG2=V, PG3=W** — **VERIFIED** (traced PWMx → NBHx/ILx → GHx/GLx → Qx → PHASE_x; TP11/12/13). IOCON: **POLH=1** (active-low HS), **POLL=0** (active-high LS), **PMOD=complementary** ; **set MCU dead-time ≈ 0** (ATA applies tCC internally; keep a tiny margin only if scope shows shoot-through).

2.2 SPI2 ↔ ATA6847 — **VERIFIED** (uses SPI2, not SPI1)

net	ATA pin	dsPIC pin	port / function
SCK	SCK 47	31	RB10 = SCK2 (RP27)
SDI (ATA → MCU)	SDO 45	32	RB11 = SDI2 (RP28)
SDO (MCU → ATA)	SDI 46	34	RC9 = SDO2 (RP42)
nCS (GPIO)	NCS 44	33	RC8 (RP41), LAT idle-high
nIRQ (GPIO in)	NIRQ 41	35	RC6 (RP39)

⚠ **Retarget the ata-esc SPI1 driver to SPI2** (SPI2CONx SFRs; pins are SPI2-native so PPS is minimal/none). Mode unchanged & **hard-won**: **MODE16=1**, **MSTEN=1**, **CKP=0**, **CKE=0**, **SMP=1**, **ENHBUF=1** (CKP=1/CKE=1 phase-shifted the word → DSR1 read **0x21** not **0x05**). BRG=14 → 3.33 MHz (drop to ~4 → 10 MHz after SI check).

2.3 Comms / debug / LED (PPS)

dsPIC pin	port	function	net	PPS
59	RD5	DShot (bidir)	DSHOT	RPn IC/OC
60	RD6	TELE_TX	TELE_TX	U_TX
61	RD4	TELE_RX	TELE_RX	U_RX
62	RD7	DEBUG_TX	DEBUG_TX	U1TX=9
63	RD8	DEBUG_RX	DEBUG_RX	U1RX
45	RC10	CAN_TX	→ ATA6561	CAN1TX
46	RC11	CAN_RX	← ATA6561	CAN1RX
30	RB7	CAN_STBY	STBY	GPIO out
39	RC0	Status LED	D4	GPIO out

2.4 Analog — ADC channels (fixed AN pins; 3 dedicated BEMF channels)

dsPIC pin	signal	net	proposed ADC channel	scale
17	BEMF_U	BEMF_U	dedicated chan (ADC1)	0.0769 div
23	BEMF_V	BEMF_V	dedicated chan (ADC2)	0.0769 div
3	BEMF_W	BEMF_W	dedicated chan (ADC1 or ADC2)	0.0769 div
6	BEMF_N (virtual neutral)	BEMF_N	dedicated chan (for option N3)	3×24k star
2	VBUS	VBUS	ADC1 chan	0.0769 → 1.92 V@25 V
1	TEMP	TEMP	ADC1 chan	10k NTC/4.7k
12	U-phase current	U_OA1_OUT	dsPIC OA1 (diff)	73 mV/A
20	V-phase current	V_OA2_OUT	dsPIC OA2 (diff)	73 mV/A
15	W-phase current	W_OA_OUT	ADC ← ATA CSA (×16)	32 mV/A
28	DC-bus current	IBUS_OA_OUT	ADC ← ATA CSA (×16)	32 mV/A

2.5 Clock / reset / power

pin	function
40	OSC1/CLKI ← Y1 8 MHz MEMS
—	MCLR (J1.12, PWRGD-gated)
5/19/26/38/48/58	VDD 3.3 V
11	AVDD (FB3 120R)
53	SWVDD (internal buck in)
56	VDDCORE ~1.1 V (do not drive)
54	LX (buck switch)
55	SWVSS

3. ADC architecture — the "3 dedicated BEMF channels" design

Key change vs the AK128 source. On AK128MC106 the code muxes **Phase A and Phase C onto AD2CH0** and pays a **ZC_AD2_SETTLE_SAMPLES** penalty + discards the post-switch sample (**HAL_ADC_SelectBEMFChannel** , **bemfSampleValid**). On the 64-pin 506 we have enough analog inputs to give **each BEMF phase its own ADC channel** that samples **every PWM trigger, simultaneously**:

```

ADC1 - CH_a  ← BEMF_U (pin17)    all three triggered by the SAME
ADC2 - CH_b  ← BEMF_V (pin23)    PG1 trigger → 3 phase voltages
ADC1 - CH_c  ← BEMF_W (pin3)     captured at one instant, no mux, no settle
(+)  CH_n    ← BEMF_N (pin6)     real star-point, for neutral option N3
(+)  Vbus, pot/throttle, currents on remaining channels

```

Consequences for the porting: 1. **Delete the A/C mux path** (`HAL_ADC_SelectBEMFChannel` , `bemfSampleValid` , `ZC_AD2_SETTLE_SAMPLES`) — replace with three static channel reads each ISR. 2. **Computed neutral** ($(V_u+V_v+V_w)/3$) **becomes trivially correct** because all three are sampled at the *same* instant (the AK128 mux made a true simultaneous 3-phase sum impossible — a major reason it used the duty model). This unlocks neutral option **N1** below. 3. Each ADC channel has its **own digital comparator** (`ADCMPx` , `CMPMOD` , `CMPLO`) — so per-phase hardware-comparator ZC (option Z2) can run on the *actual* floating phase without re-muxing a shared comparator. 4. **ADC trigger point** (carried from AKESC): center-aligned PWM, `PG1TRIGA`. Keep the dual capability: - **mid-ON center** sample for steady-state ZC vs computed/duty neutral, and - **freewheel/OFF-center** sample for the falling-polarity SW ZC (the AKESC `FEATURE_HWZC_FALLING_SW` path). The 506's spare channels let us schedule **both** an ON-window and an OFF-window conversion per phase per PWM cycle (two trigger sources, `PG1TRIGA` / `PG1TRIGB` as the ata-esc tree already does for MID-ON/MID-OFF).

4. BEMF / ZC strategy — selectable sensing back-ends

The single most important porting decision for this board is that GarudaESE physically supports **every** ZC method. We implement them behind one compile-time (and where cheap, run-time) selector so the same firmware can be A/B-tested on the bench.

```

/* garuda_config.h */
#define ZC_SRC_ATA_CMP      0 /* ATA6847 internal BEMF comparators (GPIO) */
#define ZC_SRC_ADC_CMP     1 /* dsPIC per-channel ADC digital comparator */
#define ZC_SRC_ADC_SW      2 /* dsPIC ADC software threshold (poll in ISR) */
#define BEMF_ZC_SOURCE      ZC_SRC_ADC_SW /* default for bring-up */

/* Neutral reference model (applies to ZC_SRC_ADC_*) */
#define ZC_NEUTRAL_COMPUTED 0 /* N1: (Vu+Vv+Vw)/3 from 3 simultaneous samples */
#define ZC_NEUTRAL_VDC_HALF 1 /* N2: Vbus/2 + per-phase offset (duty model) */
#define ZC_NEUTRAL_EXTERNAL 2 /* N3: real virtual-neutral pin BEMF_N (pin 6) */
#define ZC_NEUTRAL_MODE     ZC_NEUTRAL_EXTERNAL

```

4.1 The options

ID	Method	Neutral	dsPIC resource	Pros	Cons / watch-outs
Z1	ATA6847 internal comparators (BEMFEN=1)	ATA's internal ref	3 GPIO inputs + (ata-esc) PTG/poll or CLC+IC	proven in ata-esc; HW edge-blanking (EGBLT); zero ADC load	VERIFIED routed: <code>ATA_BEMF_U/V/W</code> = ATA comparator BEMF1/2/3 → MCU RA9/RA10/RB9 (digital GPIO) and TP8/7/6. So Z1 is directly available (port the ata-esc path). Comparator ref not externally trimmable.
Z2	ADC per-channel digital comparator	any of N1/N2/N3 (loaded into <code>CMPLO</code>)	3 ADC channels w/ <code>ADCMP</code>	true floating-phase signal; 1-conversion latency; programmable threshold; no mux	needs threshold update each ISR; comparator sees switching ripple → keep blanking
Z3	ADC software threshold (poll)	any of N1/N2/N3	3 ADC channels	most flexible; per-phase gain/offset; falling-SW OFF-center trick; ML-ready	24 kHz sample-rate ceiling (~90 k eRPM/phase before spp<5) — same limit AKESC documented

4.2 Neutral models (for Z2/Z3)

ID	Neutral	Formula	When it wins	Notes
N1	Computed star	$(V_u+V_v+V_w)/3$	mid/high speed, balanced motor	now correct because 506 samples all 3 simultaneously (was impossible on AK128 shared mux). Needs 3 channels read same trigger.
N2	Vbus/2 + offset (duty model)	$(V_{bus} \cdot \text{duty}) / ZC_DUTY_DIVISOR + \text{offset}[\text{phase}]$	low-speed / startup hand-off	this is the current AKESC default ; duty-dependent; per-phase <code>ZC_PHASE_OFFSET_*</code> . Beware duty positive-feedback (AKESC used measured BEMF amplitude, not raw duty, for filter-comp).
N3	External virtual neutral	sample <code>BEMF_N</code> (pin 6), compare each phase to it	when the motor/star is non-ideal; cleanest physical reference	GarudaESE has a real star-point ($3 \times 24k \rightarrow R68\ 680R + C63\ 6800pF$). Δ Neutral filter $\tau \approx 4.26\ \mu s$ vs phase $\tau \approx 1.85\ \mu s$ (2.3× lag) — must time-align or compensate (see PinMap §4).

4.3 Recommended bring-up order

- Z3 + N2** (duty model) — closest to the known-good AKESC behaviour → fastest first spin.
- Z3 + N3** (real neutral) — exploit the hardware star-point; measure the τ -lag, add comp.
- Z3 + N1** (computed) — validate the now-correct simultaneous 3-phase sum.
- Z2** (ADC comparator) with the winning neutral — push the speed ceiling.
- Z1** (ATA comparator) — only if TP6/TP7 are routed to GPIO; compare against Z2.

5. Current sensing, overcurrent & chop strategy

GarudaESE has a **mixed 4-shunt** sense (PinMap §5): U/V via dsPIC op-amps (73 mV/A), **W + DC-bus via ATA6847 CSAs** ($\times 16$, 32 mV/A, 1.65 V offset, inverted). Three independent chop levers exist:

Lever	Mechanism	Where	Best for	Caveat
C1 — Software bus chop	sample <code>IBUS_OA_OUT</code> in ADC ISR → fold-back duty	firmware	run-time current limit / soft ramp	bus shunt reads ~0 at the freewheel valley at low speed → blind to phase inrush (proven on AKESC). Sample in the mid-ON window and/or scale by duty.
C2 — dsPIC CMP3 → CLPCI	analog comparator on a current node → cycle-by-cycle PWM PCI fault	<code>hal_comparator.c</code> (port from AKESC)	hard HW ceiling, fast	threshold ≠ real amps until bias-calibrated; same low-speed bus-blindness if fed the bus shunt; feed a phase node if possible
C3 — ATA6847 native ILIM	<code>ILIMCR</code> / <code>ILIMTH</code> cycle-by-cycle chop inside the gate driver	<code>hal_ata6847.c</code>	true per-leg current limit, offloads MCU	ata-esc disabled it (capped eRPM with props at 24 V: 15–25 A commutation transients tripped ILIM). Use only for current-limited startup , then raise/disable for run.

Plan-A chop policy: - Keep **C3 (ATA ILIM)** *available* with **independent start vs run thresholds**: low `ILIMTH` during align/OL-ramp (bounds spin-up inrush — the thing the AKESC bus-chop could not see), then raise/disable at CL entry so it doesn't cap eRPM. - Keep **ATA SPCPR** (VDS short-circuit, SCFLT=15/7.5 μs, SCTHSEL=7/2000 mV) **always on** as the hard safety net (ata-esc values are 24 V-tuned). - Use **C1 software bus chop** only for telemetry-grade limiting / fold-back, sampled mid-ON. - **C2** optional, second iteration, fed from a phase shunt (not the bus) to fix the low-speed blindness.

6. Commutation / sector-PI engine — decision

Keep the AKESC `hwzc.c` PLL + `bemf_zc.c` rather than the ata-esc `sector_pi.c` + PTG path, because: - AKESC's hwzc is the more mature, bench-proven engine (defensive PI, falling-SW ZC, timing advance decoupled from speed cap, ABS_FLOOR overspeed guard, 260 k bench parity). - It is ADC-centric, which matches GarudaESE's real-divider BEMF + the 3-channel design. - The ata-esc PTG/CLC/IC chain was built specifically around the **ATA comparator GPIO** (option Z1); we only adopt it if we choose Z1.

Borrow from ata-esc only: the SPI/ATA driver, the active-low/internal-DT PWM polarity setup, and the GDU start/stop sequencing.

6.1 Clean 6-step — no FOC parameter file

The things hwzc "borrows from FOC" are **motor/board constants, not FOC algorithm params** — they only *live* in the FOC struct today. For a clean 6-step build we **extract just those into a new `motor_profile.h` and drop `garuda_foc_params.h` entirely**:

```
/* motor_profile.h — the ONLY constants 6-step needs */
typedef struct {
    uint16_t keUvSRad;    /* back-EMF constant λ/Ke (μV·s/rad) — was focKeUvSRad    */
    uint8_t  polePairs;   /* commutation table + eRPM→rad/s                          */
    /* board/timing (could also live in a board header): */
    /* VBUS_SCALE_V_PER_COUNT, LOOPTIME_TCY, timing-advance constants */
} MotorProfile;
```

Symbol (today in FOC struct)	What it really is	Clean 6-step
<code>focKeUvSRad</code>	motor back-EMF constant	keep → rename <code>keUvSRad</code> (feedforward <code>P_ff = 181.380·λ/(Vbus·dutyFrac)</code>)
<code>motorPolePairs</code>	motor property	keep
<code>VBUS_SCALE_V_PER_COUNT</code>	board scaling	keep (board header)
<code>LOOPTIME_TCY</code> , timing-advance consts	timing	keep
<code>Rs</code> , <code>Ls</code> , current-loop Kp/Ki, observer gains	FOC-only	drop — added back with FOC's own param file later

Result: a self-contained 6-step codebase with **no FOC baggage**. FOC (Plan B) brings its own param file and extends `MotorProfile` — nothing to carry now.

7. PWM / boot / start-stop sequencing (from ata-esc, validated)

Init order (main): clock → port_config (TRIS/PPS/ANSEL) → UART → `SPI_Init` → `ATA6847_Init` → verify `DSR1` (0xFF ⇒ SPI error) → PWM init (outputs disabled) → ADC init → GSP.

Start (GDU power-up): `ClearFaults` → `EnterGduNormal` (CSA enable → Standby → Normal → poll `DSR1.GDUS` , capped 2000, 0xFFFF-guarded; retry once) → enable PWM outputs → `ChargeBootstrap` → ~20 ms settle → `ClearFaults` → enable OPA + ADC ISR → align/OL-ramp → CL.

Stop: sector stop → PWM outputs off → ADC ISR off → OPA off → `EnterGduStandby` (GDU OFF + CSA off).

ATA register baseline (carry ata-esc values, re-confirm for 506 sensing): WDT off; `GDUCR1.BEMFEN` = 1 only if using Z1 (else evaluate 0); `SCPCR=0x7F` ; `GDUCR2` EGBLT (HW blank, relevant to Z1); `GDUCR3` slew 12.5 %/adaptive DT; ILIM masked from nIRQ; `CSCR` gain 16 + CSA enable for W/bus; `DOPMCR` → Normal w/ verify+retry.

7b. Control inputs — arm switch + throttle (verified against the connectors)

No analog/ADC pin reaches any connector (every J1 signal pin is digital-only; PGC/PGD = RC2/RC5 have no ADC). So Plan A uses: - **Throttle: commanded over GSP on the DEBUG UART (RD7/RD8)** — the *same* UART the board bring-up uses (confirm instance, HW sheet Q-D4). The existing broker / MCP / GUI already drive throttle; no analog pot needed. - **Arm: a digital switch** on a J1 pin — default **DShot RD5 (pin 59)** as a GPIO input (internal pull-up, switch → GND). Carry the proven button-toggle arm/stop behaviour (arm on press, stop on next press, stays stopped). - **Physical pot:** re-verification found a **reserved analog net `Speed` on pin 16 (RA6, AD3AN2)** — likely the intended throttle/pot input. **If the designer confirms `Speed` reaches a connector/pad/test-point, that becomes the pot input** (set `THROTTLE_SOURCE = ADC_SPEED`); otherwise fall back to GSP, or rework to RB3/RB6 (free `AD4AN3` / `AD4AN1` no-connect pads).

8. Build / feature-flag plan

Flag	Default (Plan A)	Purpose
MOTOR_TYPE	SIX_STEP	clean 6-step build; FOC code + param file absent (added in Plan B)
THROTTLE_SOURCE	GSP_UART	throttle over DEBUG UART/GSP (no analog pot)
ARM_INPUT	GPIO_RD5	digital arm switch (button toggle)
BOARD	GARUDAESE_EV60Y51A	selects port_config + ADC map
BEMF_ZC_SOURCE	ZC_SRC_ADC_SW	\$4 selector
ZC_NEUTRAL_MODE	ZC_NEUTRAL_VDC_HALF → then EXTERNAL	\$4.2
FEATURE_ATA_ILIM_STARTUP	1	C3 current-limited startup, independent start/run threshold
OC_CMP3 / FEATURE_HW_OVERCURRENT	0 initially	C2, enable iteration 2 (phase-fed)
FEATURE_GSP	1	live params/telemetry (mind EEPROM overlay gotcha)
FEATURE_LIVE_TUNE	1	ISR-live 6-step knobs without reflash
FCY / clock	per Y1 8 MHz + PLL	clock.c

EEPROM overlay gotcha (carry from AKESC): with `FEATURE_GSP=1`, runtime reads `profileDefaults[MOTOR_PROFILE]` + `EEPROM` — code `#define`s are inert. Bump the defaults-signature on profile change or use `FEATURE_PARAMS_FORCE_DEFAULTS` during bring-up.

9. Test phases (bench-gated, each with entry → pass criteria)

Safety: current-limited PSU, prop OFF until T4, scope on one phase + bus shunt, GSP telemetry recording per run.

Phase	Goal	Setup	Pass criteria
T0 — Power & comms	rails, clock, UART, GSP link	board on bench PSU, no motor	3.3/5/1.1 V good; DEBUG_TX banner; GSP INFO handshakes
T1 — SPI / ATA alive	SPI mode correct, ATA configured	read <code>DSR1</code> , <code>GDUCR1</code> , <code>CSCR</code> , <code>DOPMCR</code>	<code>DSR1 != 0xFF</code> ; <code>DOPMCR&7==7</code> ; registers read back as written (catches the CKP/CKE phase bug)
T2 — GDU + gate integrity	gates switch, no shoot-through	GDU Normal, manual 6-step override at low duty, no motor , scope GH/GL	clean complementary edges; internal dead-time present; <code>DSR1.GDUS</code> set; no SCP fault
T3 — Sensing sanity	ADC scales, BEMF, neutral, currents	spin motor by hand ; log BEMF_U/V/W, BEMF_N, Vbus, Ibus, Iu/Iv	3 BEMF sinusoids 120° apart; $(V_u+V_v+V_w)/3 \approx \text{BEMF}_N$ (validates N1 vs N3 + τ-lag); current zero-offset sane
T4 — Open-loop spin	align + forced commutation	low duty, motor free shaft, no prop	smooth OL ramp to ~3–4 k eRPM, bounded current; no SCP/UV
T5 — ZC back-end A/B	each ZC option detects	run Z3+N2 , then Z3+N3 , Z3+N1 , Z2 ; record per-sector miss telemetry	each option transitions OL → CL and holds idle; compare reject%/miss-by-sector; pick winner
T6 — Closed-loop sweep	throttle range, sync integrity	winner ZC, pot 0 → full → 0	sync held both directions; idle clean; ceiling characterized; zero phantom on decel
T7 — Chop / protection	startup current-limit + safety	enable C3 start-limit; force stall/over-throttle	inrush bounded at start; ILIM doesn't cap run eRPM; SCPCR catches a deliberate fault; OC telemetry true
T8 — Loaded / prop	real load, thermals, top speed	prop on, ramp to full	holds sync under load; temp in range; no BOARD_PCI at the cap (maxCLerpm ≥ natural top — carry the cap-slam lesson)
T9 — Soak / regression	endurance + corner cases	repeated arm/spin/stop, rapid pot-zero	no restart cycles; no desync; GSP corpus archived

10. Risks & open items

- 1. **ATA_BEMF_U/V/W appear to land on test-points (TP6/TP7), not MCU GPIO** → option Z1 may need a rework/strap; default to Z3/Z2 which use the real dividers. **Confirm on the schematic net + PCB.**
- 2. **dsPIC-side SPI RPN pins not yet read** (below render resolution) → confirm from the page-4 symbol before writing PPS. (ATA-side NCS44/SCK47/SDI46/SDO45/NIRQ41 are confirmed.)
- 3. **Neutral filter τ asymmetry** (4.26 μs N vs 1.85 μs phase) → N3 needs time-alignment/comp.
- 4. **Bus-shunt low-speed blindness** → don't rely on C1 for startup inrush; use C3 (ATA ILIM) start-limit.
- 5. **Mixed current-sense scaling** (73 mV/A dsPIC vs 32 mV/A ATA, inverted, 1.65 V offset) → per-channel calibration; the OC math must use the right scale per channel.
- 6. **PWM-gen → phase mapping** (PG1=U?) and **PWM → ATA polarity** must be bench-verified at T2 before any closed-loop.
- 7. **EEPROM overlay** shadowing code defaults — handle the signature/force-defaults during bring-up.

11. Forward-compatibility for FOC (Plan B, later)

- 6-step ships **without** the FOC param file; FOC adds its own (`Rs` / `Ls` / current-loop/observer) and extends `MotorProfile` (§6.1). The shared λ/Ke + pole-pairs already live in `MotorProfile`, so no migration.

- The **3 dedicated ADC channels + mid-ON current windows** are exactly what FOC needs (simultaneous phase-current sampling). The ATA CSAs (W + bus) + dsPIC OA (U/V) give 3-phase current — enough for FOC current control.
- The ATA driver, PWM (center-aligned, complementary, internal DT), and clock are FOC-ready unchanged; only the control core (Clarke/Park/SVM/observer) is added, selected by `MOTOR_TYPE`.

Generated for Project Garuda — GarudaESE / EV60Y51A bring-up. Pair with `GarudaESE_PinMap.md`.